



AI with Rav

Learn AI the way you actually think.



DAY 3 of 30

OPEN SOURCE CONTRIBUTION

"Your Fix Is Right. It Is Also Incomplete."

WHAT YOU'LL LEARN TODAY

- › What to do when a reviewer finds a gap
- › How to ask about scope before widening a PR
- › Why one fix became two PRs
- › Closing a bug that sat open for a year



"Your Fix Is Right. It Is Also Incomplete."

In One Line: My last fix worked for compressed images and silently missed uncompressed ones. The maintainer spotted it. What happened next is the most useful thing in this series.

Where we left off

Day 2's PR fixed 4-bit RGB image extraction. It was merged. It was correct.

It also only ran for **some** images — and I did not know that when I opened it.

How the gap was found

While reviewing, the maintainer pointed at a different file of his own and asked, in effect: *does your fix handle this one?*

It did not. I reproduced it and said so:

"Good catch on both counts. On your case: you're right, and I can reproduce it. The expansion sits in `_handle_flate`, which only runs for `FlateDecode/RunLengthDecode`, so an unfiltered image falls through to the plain `_image_from_bytes` call at the bottom and the raw 4bits mode goes straight to Pillow."

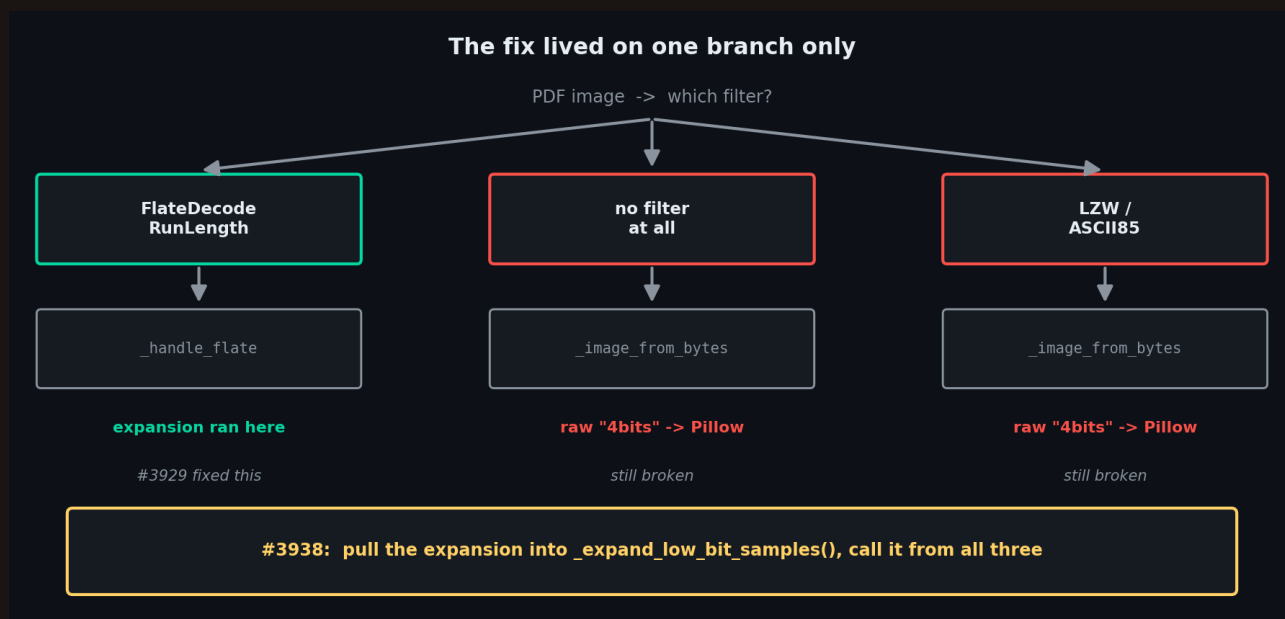
That reply is the whole lesson of this video. Three parts, in order: **he is right, I reproduced it, here is the mechanism.**

What I did not do: argue, explain why it was out of scope, or say "that's a separate issue" before checking. Reproduce first. You cannot negotiate scope on a bug you have not confirmed.

Why the fix had a hole

The expansion code lived inside a function called `_handle_flate` — the handler for **FlateDecode**, PDF's zlib compression.





The fix lived on one branch. Two other branches reached the same destination without it.

An image with no filter at all, or one using the LZW/ASCII85 fallback, never enters that function. It goes straight to `_image_from_bytes` with the raw "4bits" mode string — and Pillow raises the same error we just fixed.

The fix was in the right place for the reported case and the wrong place for the general one.

The question I asked

Here is where most contributors go wrong. The obvious move is to widen the open PR and fix everything at once. Instead I asked:

"Want me to do that in this PR, or keep this one to the flate fix and handle the inline-image path separately? Happy either way — the second one touches more paths so I didn't want to widen the scope without asking."

Then I stopped and waited.

Why asking beats deciding

A maintainer reviewing a focused PR is checking one thing. A maintainer reviewing a PR that grew mid-review has to re-check everything, including what they already approved.

You do not get to make that call for them. It is their review time and their release.

He replied with a question of his own — whether the parameters I had added would still be relevant to a general fix. I answered that they would: `colors=` and `scale=` do the actual unpacking work, and only the dispatch moves. Answer the technical question directly; that is what he is deciding on.



The outcome: two PRs

We kept them separate.

PR	Scope	Fixes
#3929	The FlateDecode path only	#3924 — the reported crash
#3938	Pull expansion out, call it from all three paths	#3367 — open since July 2025

The follow-up closed an issue that had been **open for more than a year**.

The follow-up fix

The change is almost boring, which is the point. Take the code out of the branch it was trapped in:

```
def _expand_low_bit_samples(mode, size, data, color_space):  
    """Expand 2/4-bit samples to 8-bit. Returns (mode, data)."""  
    ...
```

Then call it from the three places that build an image from raw bytes. **_handle_flate** behaves exactly as before. **bits2byte** is untouched.

+74/-15, and the merge came the day after it opened. A follow-up PR from someone who already shipped the first half is the easiest review a maintainer gets all week.

The rebase nobody warns you about

While the PR was open, an unrelated change landed on **main** that reordered the very function I was editing. My branch no longer merged cleanly.

This is normal on an active repo and it is not a setback. What matters is how you report it:

"Rebased on main — the conflict was with the **_handle_flate** reorder in #3904. Kept the new ordering and moved the low-bit branch into it; the **/Indexed** unpacking has to stay above it since the branch reads **color_space**."

Three things in one comment: **what I rebased onto**, **what conflicted**, and **the one ordering constraint I had to preserve**.

- Rebase onto their **new** structure — do not restore your old ordering because it was yours
- Say **which PR** caused the conflict, so the reviewer can see it was not your mistake



- Call out any **constraint** you had to respect, so they can check your reasoning rather than re-derive it

Never resolve a conflict by taking your side wholesale. The other change was reviewed and merged; yours has not been. Theirs wins by default, and you move your change into their shape.

The test

```
# an uncompressed 4-bit /DeviceRGB image - no filter at all
# fails on main with: ValueError: unrecognized image mode 4bits
assert img.image.mode == "RGB"
```

Same shape as before: built in the test, offline, and it fails on **main** with the precise error the maintainer described.

What to copy

- **Reproduce before you respond.** "You're right, and I can reproduce it" is the strongest sentence in code review.
- **Ask before widening.** Scope is the maintainer's decision, not yours.
- **Two clean PRs beat one sprawling one.** Each stays reviewable, and each closes its own issue.
- **A follow-up is the cheapest merge you will ever get.** You have context and trust. Use it.
- **Look for the general case yourself** — after the specific one is merged, not during.

The real result was not two merged PRs. It was that a maintainer now knows I will reproduce his case, answer his actual question, and not quietly expand a PR he already reviewed.

What is next

Both fixes needed a test that fails without them. Next video is about contributing tests alone — no bug fix at all — and why maintainers merge them faster than you would expect.

